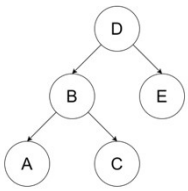


Binary Search Trees

Árboles de Búsqueda Binaria

1

1

Repaso

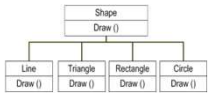
2

2

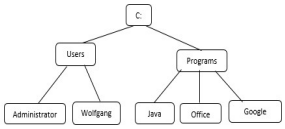
TREE ADT

Algunos datos que se pueden representar con un árbol:

- **Directorios**
- **Jerarquía de clases**
- **Árboles genealógicos**



C:
Users
Administrator
Wolfgang
Programs
Java
Office
Google



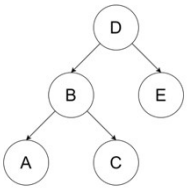
3

3

Repaso

Un repaso rápido a la terminología:

- Un árbol se compone de subárboles.
- Cada nodo es la raíz de un árbol/subárbol.
- Por ejemplo, en la figura tenemos que:
  - D es la raíz de todo el Árbol.
  - B es la raíz del subárbol que contiene B, A y C.
  - A, C y E, son cada una raíz de subárboles de tamaño 1.



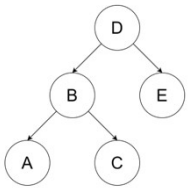
4

4

Repaso

Un repaso rápido en el recorrido:

- Cada movimiento a través del Árbol es desde la perspectiva de la raíz actual.
- Ejemplo, para el recorrido en orden (in order):
  - Aplicamos las acciones (Izquierda, raíz, Derecha) en cada nodo.
  - Comenzamos en D (L, Ro, R). Nos movemos a la izquierda a B.
  - En B (L, Ro, R). Nos movemos a la izquierda a A.
  - En A (L, Ro, R). No podemos movernos a la izquierda.



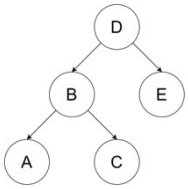
5

5

Repaso rápido

Un repaso rápido en el recorrido:

- Ejemplo, para el recorrido en orden (in order):
  - En A (L, Ro, R). Hacemos la acción en la raíz.
  - En A (L, Ro, R). No podemos movernos a la derecha.
  - En A (L, Ro, R). No queda nada por hacer, volver a B.
  - En B (L, Ro, R). Hacemos acción en la raíz.
  - In B (L, Ro, R). Nos desplazamos a la derecha hacia C.
  - In C (L, Ro, R). No podemos movernos a la izquierda.
  - Y así sucesivamente.....



6

6

Binary Search Trees

Árboles de Búsqueda Binaria

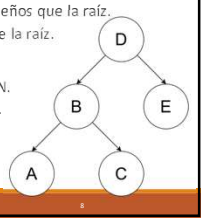
7

7

Árbol de Búsqueda Binaria
(The Binary Search Tree ADT)

¿Qué es un árbol de búsqueda binaria (BST)?

- Es un árbol binario desbalanceado con estas propiedades:
  - Todos los nodos del subárbol de la izquierda son más pequeños que la raíz.
  - Todos los nodos del subárbol derecho son más grandes que la raíz.
- En otras palabras, para cualquier nodo N:
  - Todos los descendientes de la izquierda son menores que N.
  - Todos los descendientes de la derecha son mayores que N.

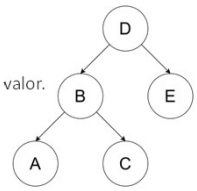


8

8

Árbol de Búsqueda Binaria
(The Binary Search Tree ADT)

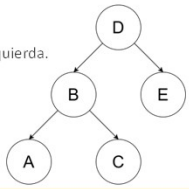
- Esta estructura utiliza pares key-value.
- Al igual que en Maps, no hay keys duplicados.
- El orden se establece por comparación de keys.
- Para la mayoría de los casos, asumimos que key == valor.



9

Árbol de Búsqueda Binaria
(The Binary Search Tree ADT)

- Este árbol es ideal para buscar, ya que podemos descartar un subárbol con cada movimiento.
- Al igual que la búsqueda binaria.
- Buscamos de la siguiente manera:
  - Si el key es más pequeño que el key actual, muévase a la izquierda.
  - De lo contrario, muévase a la derecha.
  - Se detiene cuando se encuentra el key o un valor nulo.

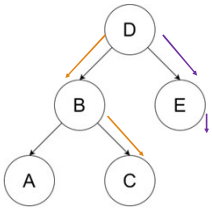


10

Árbol de Búsqueda Binaria
(The Binary Search Tree ADT)

Ejemplo 1: Buscamos C.
- Comenzamos en D.
- C < D, Nos movemos a la izquierda.
- C > B, Nos movemos a la derecha.
- C == C. EncontramosC!!!

Ejemplo 2: Buscamos F.
- Comenzamos en D.
- F > D, Nos movemos a la derecha.
- F > E, Nos movemos a la derecha.
- Llegamos a null. Deducimos que F no está en el árbol.

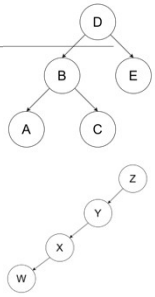


11

Árbol de Búsqueda Binaria
(The Binary Search Tree ADT)

Debido a esta estructura de árboles, podemos garantizar que:

- Una vez que vamos a un subárbol, podemos descartar el otro por completo. El key no puede estar en el otro subárbol.
- El key **no** está en el Árbol una vez que llegamos a un callejón sin salida. El key **no** podría estar en otro lugar.

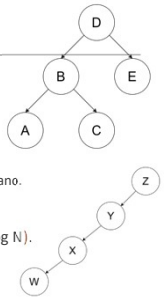


12

Árbol de Búsqueda Binaria
(The Binary Search Tree ADT)

Complejidad:

- Términos:
  - H – Altura del árbol
  - N – número total de nodos en el árbol. (tamaño)
- La complejidad de sus operaciones es O(H).
  - Comenzamos en la raíz y encontramos el nodo en su hijo más lejano.
- Si H < N, entonces el BST es eficiente.
- Por lo general, H = log N. En promedio, nuestro límite es O(log N).

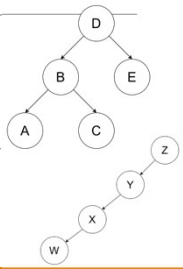


13

Árbol de Búsqueda Binaria
(The Binary Search Tree ADT)

Complejidad:

- El peor de los casos es O(N), cuando H = N.
- Esto es cuando el árbol es muy pequeño o está demasiado desequilibrado.
- Ejemplo: Busque W en el árbol en la parte inferior derecha.
  - Debemos movernos a través de todos los N nodos para llegar a él.

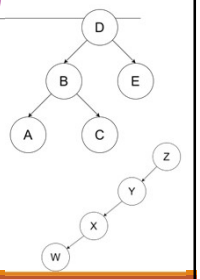


14

Árbol de Búsqueda Binaria
(The Binary Search Tree ADT)

Complejidad:

- Cuanto más equilibrado sea el BST, mejor debe ser su complejidad temporal.
- La raíz del árbol afecta el equilibrio del árbol.
- En la segunda figura
  - Empezamos en la Z.
  - Todos los demás valores serán más pequeños.
- El orden de adición a este árbol es importante.



15

BST Implementación

16

Diferencias con árboles

- Tenga en cuenta que el BST, a diferencia del BinaryTree, tiene un método add().
- Dado que el BST tiene una estructura más específica, podemos construirlo agregando nodos uno por uno.
- Ya no es un grupo de subárboles que simplemente se agregan unos a otros.

17

17

Adición de nodos

¿Cómo añadimos nodos?

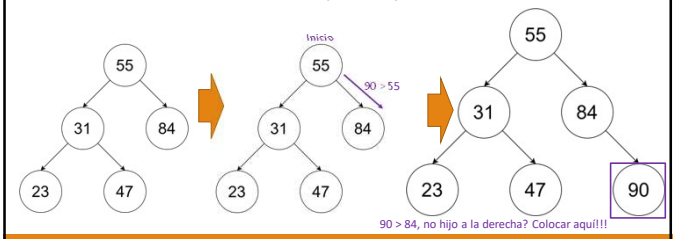
- Buscamos dónde se debe colocar el key.
- Cuando encontramos un callejón sin salida, lo añadimos allí.
- Si se encuentra el key, dejamos de buscar y regresamos. No hagas nada. Al igual que en Sets.

18

18

Adición de Nodos

Add(90, 90);

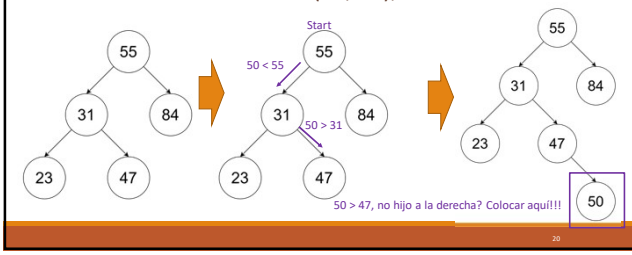


19

19

Adición de Nodos

Add(50, 50);

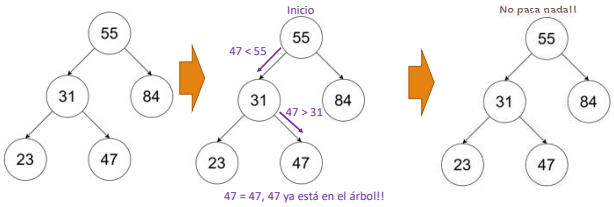


20

20

Adición de Nodos

Add(47, 47);



21

Encontrar nodos

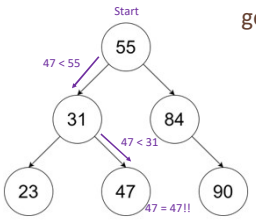
¿Cómo conseguimos las Entradas?

- Buscamos dónde debe colocarse la clave (key).
- Cuando encontramos un callejón sin salida, regresamos nulo.
  - La entrada no está en el BST.
- Si se encuentra la clave (key), devuelve el valor.

22

Encontrar nodos

get(47);



Devuelve (47, 47)

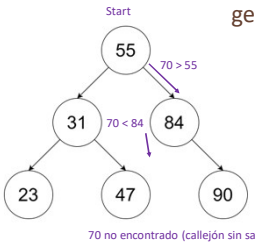
Buscamos la clave (key) dada.

- Si se encuentra, devolvemos su valor.
- Si no, devuelvo null.

23

Encontrar nodos

get(70);



Devuelve null

Buscamos la clave (key) dada.

- Si se encuentra, devolvemos su Entrada.
- Si no, devuelvo null.

24

Eliminación de nodos

¿Cómo eliminamos los nodos?

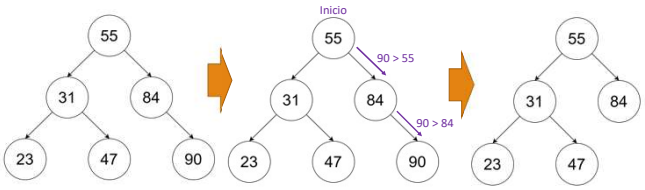
- Buscamos dónde se debe colocar el key.
- Cuando encontramos un callejón sin salida, devolvemos null.
- Si se encuentra el key, se puede realizar una de estas tres acciones:
  - Si el nudo es una hoja, elimínalo.
  - Si el nodo tiene 1 hijo, reemplácelo por su hijo.
  - Si nodo tiene 2 hijos (el caso más difícil), reemplazarlo por su predecesor en orden (in order).

25

25

Eliminación de nodos

remove(90);



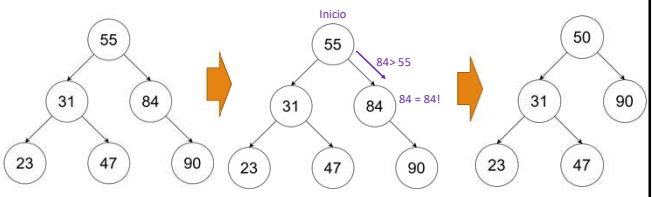
¡90 es una hoja! Simplemente quítalo.

26

26

Eliminación de nodos

remove(84);



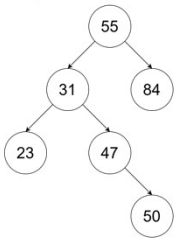
84 solo tiene un hijo. Podemos poner al 90 en la posición del 84.

27

27

Eliminación de nodos

remove(55);



55 tiene 2 hijos; debe ser reemplazado por su predecesor en orden (in order).

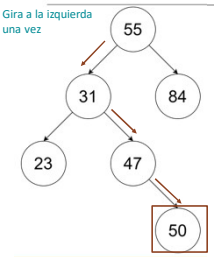
- ¿Qué significa eso?
- El recorrido en orden(in order) de este árbol es:
  - 23, 31, 47, 50, 55, 84
- 55 debe ser reemplazado por 50.
- Es el nodo que lo precede cuando recorremos en orden (in order).

28

28

Eliminación de nodos

Gira a la izquierda una vez



remove(55);

- 55 debe ser reemplazado por 50.
- ¿Necesitamos hacer el recorrido en orden (in order) para encontrar el nodo a eliminar?

No realmente

- Para encontrar al predecesor en orden (in order):
  - Muévete al subárbol izquierdo del nodo a ser eliminado. (Gira a la izquierda una vez)
  - Entonces encontramos el mayor descendiente de ese subárbol. (Vamos a la derecha hasta que no podamos más)

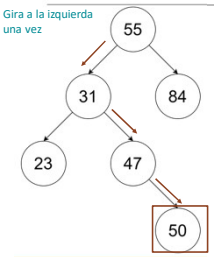
Gira a la derecha hasta llegar a un callejón sin salida. (Último hijo derecho)

29

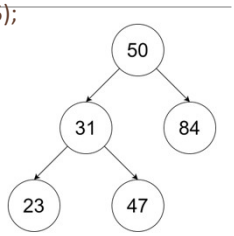
29

Eliminación de nodos

Gira a la izquierda una vez



remove(55);



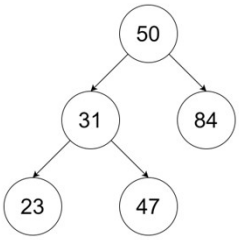
Gira a la derecha hasta llegar a un callejón sin salida. (Último hijo derecho)

30

30

Eliminación de nodos

remove(55);



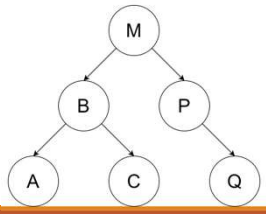
- ¿Por qué elegir el predecesor en orden (in order)?
- Como vimos, este es el descendiente mayor del subárbol izquierdo.
- Significado:
  - Este nodo es más pequeño que todo el subárbol derecho.
  - Este nodo es mayor que todo el subárbol izquierdo.
- Mantenemos el orden del BST.

31

31

Métodos sencillos

- **¿Y si quisiéramos mostrar el contenido del Árbol en orden ascendente?**
- **¿Qué desplazamiento deberíamos usar?**



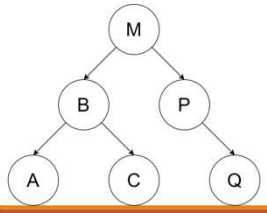
32

32

Métodos sencillos

- ¿Y si quisiéramos mostrar el contenido del Árbol en orden ascendente??
- ¿Qué desplazamiento deberíamos usar??

- **Desplazamiento In order!!!**
  - A, B, C, M, P, Q



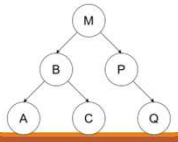
33

33

Métodos sencillos

- **Recorrido In order!**
  - A, B, C, M, P, Q

Sería diferente la implementación del recorrido de un BST que la de un BinaryTree normal?



34

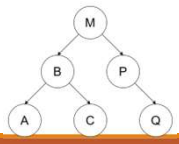
34

Métodos sencillos

- **Recorrido en orden (in order)!**
  - A, B, C, M, P, Q

¿Sería diferente la implementación del recorrido de un BST que la de un BinaryTree normal?

- No, no lo es.



35

35

Aplicaciones

36

36

Maps y Sets

Los mapas y los sets pueden implementarse fácilmente usando un BST.

37

BSTs en Java

Java no tiene una clase "BinarySearchTree" incorporada en la biblioteca estándar.

Sin embargo, Java provee clases que se implementan usando árboles de búsqueda binarios auto-equilibrados:

- TreeMap
- TreeSet

38

Qué son realmente?

Ambos se basan en un **Árbol Rojo-Negro** (un tipo de BST auto-equilibrado), por lo que mantienen el orden automáticamente.

TreeMap

- Almacena pares clave-valor (key-value)
- Las claves se mantienen ordenadas
- Buen reemplazo para un "mapa" BST

TreeSet

- Solo almacena elementos únicos
- Ordenado
- Respaldado internamente por un TreeMap

39

TreeSet y TreeMap

TreeSet y TreeMap se implementan utilizando específicamente un tipo de árbol de búsqueda binaria, un **Red-Black Tree**.

Qué significa eso

Un árbol de búsqueda binario **(BST)** sigue:

- Subárbol izquierdo → valores más pequeños
- Subárbol derecho → valores mayores

TreeSet y **TreeMap** siguen esta regla internamente, así que:

- Los elementos (o claves) siempre están ordenados
- Las operaciones utilizan la navegación por el árbol

40

TreeSet y TreeMap (distinción importante)

NO son BSTs simples.

Son:

BSTs auto-balanceados (árboles rojo-negros)

Esto significa:

- El árbol se mantiene equilibrado automáticamente
- Se mantiene la altura≈ log n
- Las operaciones son siempre **O(log n)**

41

41

TreeSet (Core)

Método	Descripción	Ejemplo
add(E e)	Añade elemento (sin duplicados)	set.add(10);
remove(Object o)	Elimina elemento	set.remove(10);
contains(Object o)	Verifica existencia	set.contains(5);
size()	Número de elementos	set.size();
isEmpty()	Checks if empty	set.isEmpty();
clear()	Removes all elements	set.clear();

42

42

TreeSets (Tree-Specific (BST-related) Methods)

Método	Descripción	Ejemplo
first()	Devuelve el elemento más pequeño	set.first();
last()	Devuelve el elemento más grande	set.last();
higher(E e)	Elemento más pequeño estrictamente mayor que e	set.higher(10);
lower(E e)	Elemento más grande estrictamente menor que e	set.lower(10);
ceiling(E e)	Elemento más pequeño ≥ e	set.ceiling(10);
floor(E e)	Elemento más grande ≤ e	set.floor(10);
iterator()	Retorna iterador	Iterator<Integer> it = set.iterator();
subSet(E from, E to)	Elementos en rango [desde, hasta]	set.subSet(10, 50);

43

43

TreeMap (Core)

Método	Descripción	Ejemplo
put(K key, V value)	añadir/actualizar	map.put(1,"A");
get(Object key)	Devolver Valor	map.get(1);
remove(Object key)	Eliminar entrada	map.remove(1);
size()	Número de valores	map.size();
isEmpty()	Comprueba si está vacío	map.isEmpty();
clear()	Eliminar todas las entradas	map.clear();

44

44

TreeMaps
(Tree-Specific Methods)

Método	Descripción	Ejemplo
containsKey(Object key)	Comprueba si existe la clave	map.containsKey(2);
containsValue(Object value)	Comprueba si existe el valor	map.containsValue("A");
firstKey()	clave más pequeña	map.firstKey();
lastKey()	Clave más grande	map.lastKey();
higherKey(K key)	La clave más pequeña > dada una clave	map.higherKey(10);
lowerKey(K key)	La clave más grande < la clave dada	map.lowerKey(10);
ceilingKey(K key)	Clave mínima ≥ clave dada	map.ceilingKey(10);
floorKey(K key)	La clave más grande ≤ la clave dada	map.floorKey(10);
keySet()	Devuelve conjunto de claves	map.keySet();
values()	Devuelve la colección de valores	map.values();
entrySet()	Devuelve pares clave-valor	map.entrySet();

45

45

Eficiencia

- Los BST son eficientes para la búsqueda en la mayoría de los casos. O(log N)
- Si están demasiado desequilibrados, la complejidad puede llegar hasta O(N).
- El orden, en el que sumamos valores, es relevante.
  - Si la raíz es demasiado pequeña o demasiado grande, un subárbol se llenará más rápido que el otro.

¿Podemos mejorar?

46

46

Eficiencia

¿Podemos mejorar?

- Sí, existen variaciones del BST que se equilibran por sí mismos.
- Esto significa que cada vez que se añade un nodo que desequilibra el árbol, mueve los nodos para volver a equilibrarlo.
- Algunos ejemplos son:
  - Árboles AVL
  - ÁrbolesBlack-Red

47

47

Conclusiones

48

48

Conclusiones

- Los BSTs son un tipo especial de árbol binario.
- Además de tener solo 2 hijos, estos Árboles tienen un orden específico.
  - Las claves (keys) más pequeñas que el nodo van a su subárbol izquierdo.
  - Las claves (keys) más grandes que el nodo van al subárbol derecho.
- Trabajamos con pares clave-valor (key-value pairs).